CipherVault: Secure File Storage with Client-Side Encryption and Role-Based Access Control

PRAMOD REDDY YALALA
Roll No. 23K91A05J9
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

PENDELA VYSHNAVI
Roll No. 23K91A05J1
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

LUKE ABRAHAM SAM
Roll No. 23K91A05D4
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

MUDAVATH SURESH
Roll No. 23K91A05G2
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

D V V DEEPTHI
[DESIGNATION]
(project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

***Abstract*---**Files kept on third-party storage are exposed to the provider, to insiders and to anyone who obtains a database backup, and access rules and audit trails kept on the same server are only as trustworthy as that server. CipherVault is a runnable web application that encrypts every file in the browser with AES-256-GCM before upload, wraps the per-file key for each recipient with RSA-2048-OAEP, enforces Owner, Editor and Viewer roles on the server, checks a SHA-256 digest on every download and records every sensitive action in a hash-chained audit log. We measured each layer on one desktop computer. Browser AES-GCM took 23.8 ms for 10 MiB; an encrypted 10 MiB upload took 209 ms against 194 ms unencrypted; a role matrix of 36 cells matched in 36; and 2,000 of 2,000 ciphertext bit flips were rejected. The audit chain caught 8 of 9 classes of tampering but missed truncation of its tail, and concurrent writers corrupted it. We report these limits and the fixes they suggest.

***Index Terms*---**access control, AES-GCM, audit log, client-side encryption, key wrapping, RBAC, secure file storage.

I. INTRODUCTION

Organisations keep sensitive documents on storage they do not control. At such a service the provider, its administrators and anyone holding a stolen backup can all read plaintext, and a network attacker can alter what is stored. Encrypting a file before it leaves the owner's machine removes the provider from the circle of trust, but implementations are easy to get wrong: the attacks on the deployed MEGA service showed that malleable key handling undoes end-to-end encryption in practice [2]. Fine-grained alternatives such as attribute-based encryption [3] move the policy into the ciphertext but are computationally heavy, and a small team sharing documents seldom needs more than three levels of access: read, write and manage.

Published designs for this problem leave a measurement gap. The declared base paper of this work [1] proposes attribute-based encryption with a blockchain for health records and evaluates it by calculation from primitive timings, not by running a system. The closest file-storage architecture [4] reports +320 ms for a 10 MB upload, 98.6% access-control accuracy and 100% audit-tamper detection, but publishes no hardware, per-operation timings or tamper methods. What is missing is a runnable, general-purpose file store that is measured layer by layer, including where it fails.

CipherVault is such a store. Its contributions are: (1) a working web application in which the browser performs all file cryptography with the Web Crypto API [5], so the server holds only ciphertext, wrapped keys, password hashes and metadata; (2) per-recipient key wrapping that lets an owner share, re-role and revoke access without ever sending a file key to the server in the clear; (3) a server-enforced Owner / Editor / Viewer model checked on every request, tested exhaustively over 36 principal-operation cells; and (4) a hash-chained audit log together with a tamper study that states what the chain does and does not detect. Section II surveys related work, Section III describes the method, Section IV gives measurements and Section V concludes.

A. System Hardware

All measurements were taken on one desktop: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11, no GPU. Browser cryptography ran in Chrome 155.0.8059.85 (headless); API timings used the loopback interface. Other jobs shared the processor during the runs, so timings are indicative and varied by up to about three times between runs at 10 MiB.

B. Software Requirements

Python 3.11.9 with FastAPI, SQLAlchemy 2, PyJWT, bcrypt and cryptography; SQLite by default and PostgreSQL through Docker; React 19 and Vite 8 for the interface, with the Web Crypto API for all browser cryptography; pytest and Playwright for tests. The code base is 1,184 lines of backend Python and 6,831 lines of front end.

II. LITERATURE SURVEY

Base paper. Nekouie et al. [1] propose sharing electronic health records through a semi-trusted cloud. ClinicalBERT extracts data attributes from clinical notes, and an elliptic-curve ciphertext-policy attribute-based scheme in the style of Bethencourt et al. [3] encrypts each record under a policy over both user and data attributes. Several key-generation centres issue partial keys, so no authority holds a user's full key, and a private Hyperledger Fabric ledger [6] stores attribute public keys and policies. The design has 23 phases and a fixed 160-bit key length. Its evaluation is analytic: total

cost is calculated from operation counts and OpenSSL primitive timings taken on an HPE server (6-core E5-2620 v3, 32 GB) and a Raspberry Pi 3 B+, giving about 8,506 µs for five attributes against 8,812 to 71,323 µs for ten compared schemes. No dataset, extraction accuracy, end-to-end upload time or role check is reported, and it is not a file-storage product. CipherVault adopts its trust model (an untrusted store that sees only ciphertext) and its plaintext-hash check, but not attribute-based encryption, the language model or the blockchain.

Related work. Storage systems for untrusted servers date from SiRiUS [7], which adds cryptographic read and write control over network file systems, Plutus [8], which groups files to cut key distribution, and Tahoe-LAFS [9], which combines capabilities with erasure coding; none of their abstracts describes a tamper-evident audit log. Recent papers use commodity primitives: a hybrid AES-256, RSA and SHA-256 store [10], Geocrypt, which adds location and time gating and Ethereum contracts [11], and the IJEDR architecture with browser-side AES-256-GCM, RSA-2048 wrapping, three roles, a chained audit log with ECDSA signatures, multi-factor sign-in and 4 MB chunking [4]. For access models, role-based control [12] grants permissions to roles rather than to users. For logs, Schneier and Kelsey [13] showed how hash linking makes earlier entries impossible to alter undetectably on a machine that is later compromised, and Crosby and Wallach [14] gave Merkle-tree logs with logarithmic proofs.

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
Base paper [1]	EC CP-ABE, ClinicalBERT, Fabric	None; HPE and Pi timings	~8,506 µs, 5 attributes (calculated)	No file store; not measured end to end
IJEDR [4]	AES-GCM, RSA, RBAC, signed chain	50 users, 900 attempts	+320 ms/10 MB; 98.6%; 50/50 tamper	No hardware or method given
Geocrypt [11]	RSA, location, Ethereum	Abstract only	Qualitative	Spoofable location; latency
Hybrid store [10]	AES-256, RSA, SHA-256	Abstract only	No figures	No roles or audit trail
MEGA attacks [2]	Cryptanalysis of key hierarchy	Deployed service	Full file compromise	A lesson, not a system
SiRiUS [7]	Crypto access over NFS/CIFS	Prototype	File-level sharing	Not a web app; no audit log described
Plutus [8]	Filegroups, efficient revocation	OpenAFS prototype	Overhead near network-encryption systems	No audit log described
Tahoe-LAFS [9]	Capabilities, erasure coding	Grid design	Provider-blind	No roles or audit log described
Schneier-Kelsey [13]	Forward-secure hash chain	Protocol	Past entries unalterable	Needs secure verifier
Crosby-Wallach [14]	Merkle log, log-size proofs	Prototype	Log-size proofs	Needs published commitments

Strengths and weaknesses. The attribute-based and blockchain designs offer fine-grained policy and no single key authority, at the cost of pairing or elliptic-curve work per record and heavy infrastructure; the capability and filegroup systems show that per-file keys scale, but they target file-system clients rather than browsers; the recent file-storage papers are the closest in aim and the weakest in evidence, since the quantities that would let a reader reproduce their results are not given. The cryptanalysis of MEGA [2] is the strongest argument for measuring and testing a design before trusting it, and it motivates the bit-flip and token-forgery tests reported here.

Research gap. The surveyed work splits into attribute-based schemes evaluated by calculation, cryptographic file systems built as prototypes, role models stated as standards, and a few recent papers that report headline outcomes without their conditions. None provides a general, runnable file store that joins browser-side AES-256-GCM, per-recipient RSA key wrapping, server-enforced roles, a plaintext-hash check and a hash-chained audit log, measures each layer on one machine and reports its own weaknesses. This paper fills that gap.

III. METHODOLOGY

A. Design and Trust Model

CipherVault has no machine-learning component and no trained dataset: it is cryptographic and access-control software, and its data are fictional demonstration files and measurement workloads. The browser is the trusted endpoint; the FastAPI server is treated as honest-but-curious for confidentiality and trusted for access decisions and logging (Fig. 1). The server therefore stores ciphertext under random 32-hex file names, public keys, bcrypt password hashes, wrapped keys and a SQL metadata and audit schema of four tables, but never plaintext or a private key.

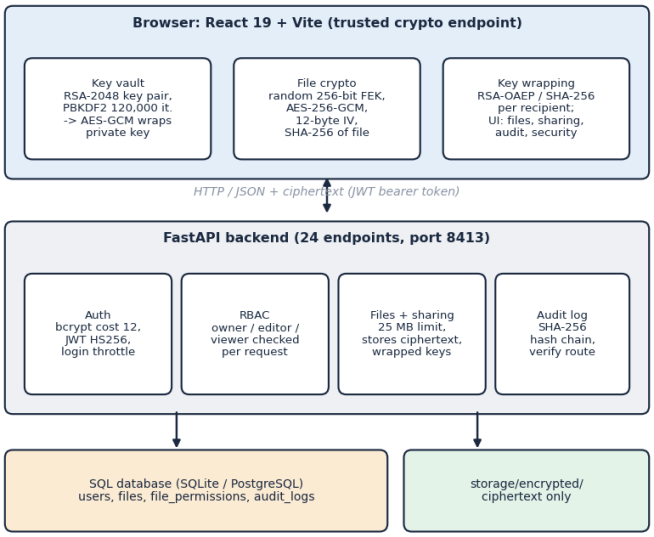


Fig. 1. System architecture of CipherVault: browser-side cryptography, FastAPI service with 24 endpoints, SQL database and a ciphertext-only store.

B. Key Hierarchy and File Encryption

At registration the browser generates an RSA-2048 key pair, derives a key-encryption key from the password with PBKDF2-HMAC-SHA-256 [15] (120,000 iterations, 16-byte random salt) and uses it to encrypt the private key with AES-GCM [16]; only the encrypted bundle and the public key are sent to the server. Each file M receives a fresh random 256-bit file key FEK and a 12-byte random IV, is encrypted in memory, and the FEK is wrapped under the owner's public key with RSA-OAEP [17]:

$$(C, T) = \text{AES-GCM}_{enc}(FEK, IV, M), \quad h = \text{SHA-256}(M) \tag{1}$$

$$W = \text{RSA-OAEP}_{pk}(FEK) \tag{2}$$

The primitives follow from the threat model. GCM [16] gives confidentiality and integrity in one pass, which a plain CBC mode would not; its security needs a fresh nonce under each key, and for that reason every upload and every replacement draws a new random 96-bit IV, a practice whose failure mode is documented for GCM in TLS [18]. RSA-OAEP is chosen over attribute-based encryption because a handful of roles needs per-recipient keys, not policies over attributes, and the wrapped key is only 256 bytes. The 120,000-iteration derivation costs 32.8 ms here; the cost is linear in the iteration count, so the 310,000 iterations of [4] would take about 84.7 ms by proportion, a calculation and not a measurement. A larger count would slow brute-force guessing of a stolen key bundle and is the first parameter we would raise.

The upload carries C with its 16-byte tag T, the IV, the wrapped key W and h. To share, the owner's browser unwraps the FEK with the private key and re-wraps it under the recipient's public key, so the server stores one 256-byte wrapped key per recipient. On download the browser unwraps its own copy, decrypts, rejects the file if the GCM tag fails and then compares SHA-256 of the plaintext with h. The server accepts at most 25 MB per file, a 12-byte IV, a 64-hex digest and a 256-byte wrapped key on share; logins pass a bcrypt [19] (cost 12) check and receive an HS256 JSON Web Token [20] valid for 120 min, with an in-memory limit of 20 attempts per 300 s per address and email.

The private key is decrypted only in browser memory after the user enters the password on the unlock screen; a page reload locks the vault again, and a wrong password makes the GCM check on the key bundle fail, so no server round trip can confirm or deny a guess. When an owner or editor replaces a file's content, the browser decrypts the old key copy, encrypts the new bytes under the same FEK with a fresh IV and uploads them, so every existing wrapped key stays valid and nobody has to be re-shared. The server never learns the FEK, so it cannot check that a replacement was encrypted under it; the next downloader detects a mismatch through the tag and the digest.

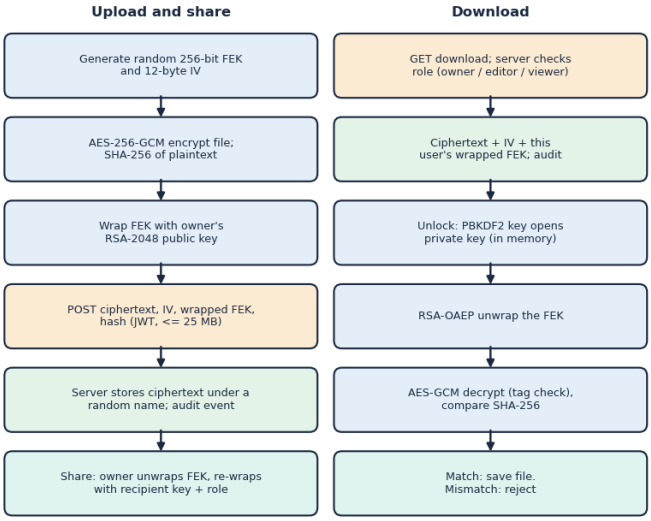


Fig. 2. Main flow: encrypt-and-share on the left, download-decrypt-verify on the right.

C. Role-Based Access Control

Every file request resolves the caller's effective role as owner, editor, viewer or none. Owner may read, replace content, delete, share, change a role, revoke and read the file's history; editor may read and replace content; viewer may read; a stranger has no access, and each refusal writes an unauthorized_access event. The check is a server-side function on every route (complete mediation), following the role model of [12]; the global account role column exists but plays no part in decisions.

The permitted-operation matrix (Table III) is derived from three predicates in the permission service: can_read for owner, editor and viewer, can_write for owner and editor, and an owner-only test for delete, share, role change, revoke and the per-file history. Listing a file's access list is open to anyone who has access. Roles are per file: the same account can own one file, edit a second and merely view a third.

D. Hash-Chained Audit Log

Each sensitive action (register, login, upload, download, update, share, role change, revoke, delete, denial, verification) appends a row whose hash binds the previous hash to the event:

$$h_n = SHA\text{-}256(\, h_{prev} \mid t_n \mid u_n \mid f_n \mid a_n \mid s_n \mid d_n \,) \qquad (3)$$

where t is the timestamp, u the user, f the file, a the action, s the success flag, d the details and h_prev of the first entry is the string GENESIS. The verify route recomputes the chain from the oldest entry. Unlike the signed or Merkle-tree logs of [13] and [14], the chain has no secret key and no external anchor, which Section IV measures. The threats addressed are a curious or compromised database or disk (only ciphertext and wrapped keys are exposed), an unauthorised account (denied and logged), a modified ciphertext (rejected by the tag and digest) and casual edits to the log. Not addressed are a malicious server that ships altered JavaScript, a compromised client device, loss of the password and an attacker with write access to the whole database.

E. Implementation

The backend exposes 24 REST endpoints in five routers (authentication, users, files, sharing, audit) over four tables: users, files, file_permissions with a unique pair of file and user, and audit_logs. Uploads are multipart forms; downloads return the ciphertext as a stream with the IV, wrapped key and digest in response headers, so the browser needs one request to decrypt. The interface has landing, sign-in, dashboard, file list, file detail with history, sharing dialog, audit, security and unlock screens, built to work down to a 360 px width. Demonstration accounts for the three roles seed fictional files on first use.

IV. RESULTS

A. Test Conditions

Measurements ran on 2026-10-22 on the machine of Section I-A. Sizes up to 1 MiB used 15 to 50 repetitions (1 KiB and 64 KiB operations were timed in batches because Chrome's clock resolves 0.1 ms), 10 MiB used 6 to 15 and 25

MiB used 3 to 4. The suite also ran 21 of 21 pytest tests (13 s), 17 of 17 API end-to-end checks and a browser walk of 18 of 18 steps; all passed. Medians are reported unless stated. The browser timings run the same primitives and parameters as the interface's crypto module in headless Chrome, the transfer timings drive the real upload and download routes with a bearer token, the role matrix registers fresh accounts and shares a real file, and the tamper study edits copies of the real audit rows and checks them with the verifier's hash rule copied into the script.

TABLE II
MEASURED RESULTS (MEDIAN MS UNLESS STATED)

Measurement	Result
AES-256-GCM encrypt / decrypt, 1 MiB	2.40 / 2.00
AES-256-GCM encrypt / decrypt, 10 MiB	23.8 / 26.2
AES-256-GCM encrypt / decrypt, 25 MiB	53.8 / 50.9
SHA-256, 10 MiB	12.9
RSA-2048 key generation / wrap / unwrap	102 / 0.030 / 0.71
PBKDF2, 120k iterations	32.8
Encrypted / plain upload, 1 MiB	63.0 / 52.9
Encrypted / plain upload, 10 MiB	209 / 194
Decrypted / plain download, 10 MiB	184 / 157
API upload / download, 10 MiB	139 / 102
API upload / download, 25 MiB	288 / 177
Login (bcrypt) / GET me / list files	308 / 6.32 / 40.3
Share / change role / revoke	37.8 / 22.7 / 24.7 (1 run)
Audit-event write (p95)	19.0 (90.0)
Role matrix cells matched	36 of 36
Ciphertext bit flips rejected	2,000 of 2,000
Audit-tamper classes caught	8 of 9

B. Cryptographic and Transfer Cost

Browser AES-256-GCM is fast: 23.8 ms to encrypt and 26.2 ms to decrypt 10 MiB, 53.8 and 50.9 ms at 25 MiB, growing close to linearly with size (Fig. 3a); SHA-256 is about twice as fast. Ciphertext is exactly 16 bytes longer than plaintext. Asymmetric work is paid once per file or user: key generation takes 102 ms, wrapping 0.030 ms, unwrapping 0.71 ms, and the 120k-iteration key derivation 32.8 ms. End to end through the real server, encryption added 10.2 ms to a 1 MiB upload (63.0 against 52.9 ms) and 14.4 ms to a 10 MiB upload (209 against 194 ms); decrypting and verifying a 10 MiB download took 184 against 157 ms plain (Fig. 3b). The overhead is a small part of the transfer but of the same size as the run-to-run spread of the shared host, so we read it as roughly 10.2 to 14.4 ms, not as a precise figure. A 25 MiB upload through the API took 288 ms and the download 177 ms; a request above the limit received HTTP 413.

The remaining server routes are light. A login costs 308 ms at the median and a failed login 326 ms, almost all of it the deliberately slow bcrypt hash; authenticated metadata calls take 6.32 ms, the health route 1.68 ms, listing 65 owned files 40.3 ms, sharing 37.8 ms, a role change 22.7 ms and a revoke 24.7 ms, each a single run. Writing one audit event took 19.0 ms at the median and 90.0 ms at p95, which is the price of the read-then-insert hash step on every request that is logged. API upload and download time grew about linearly with size, from 27.2 and 21.7 ms at 1 KiB.

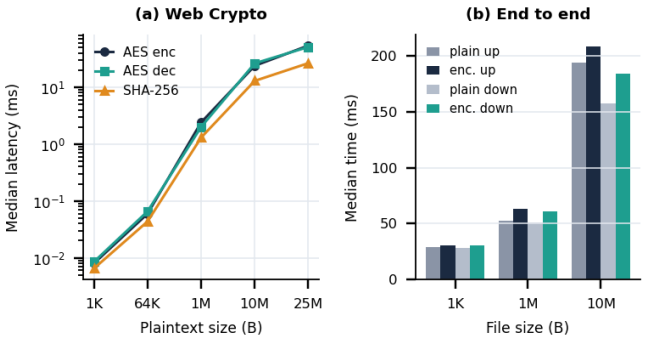


Fig. 3. Median latency: (a) AES-256-GCM and SHA-256 in the browser against plaintext size; (b) encrypted and plain transfers through the server.

C. Output Screens

Fig. 4 shows the sharing dialog of a demonstration owner with an editor and a viewer listed; the text states that the file key is re-wrapped in the browser and never sent in plaintext. Fig. 5 shows the audit page after the verify button was pressed: the chain was valid over 226 entries at that moment (a later verification of the measurement database covered 5,043 valid entries).

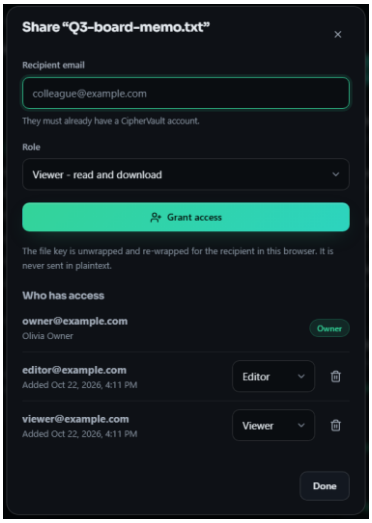


Fig. 4. Share dialog (real capture, 1440x900 page): per-file roles and the key re-wrap notice.

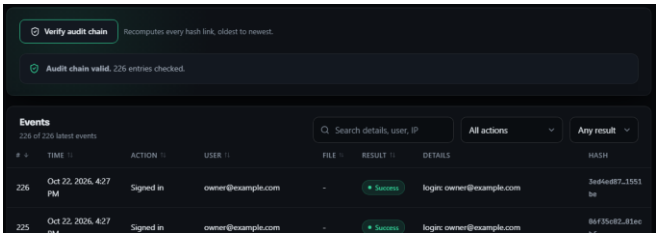


Fig. 5. Audit page after chain verification: 226 entries valid, with the hash column (crop of a real capture).

D. Access Control and Integrity

A live test of four principals (owner, editor, viewer and an unrelated user) against nine operations (metadata, download, update, list access, share, change role, revoke, file history, delete) expected 16 allowed and 20 denied cells and observed exactly those: 36 of 36; unauthenticated requests received HTTP 401, as did forged, expired, wrong-secret and

alg=none tokens; after a revoke the same user's download returned HTTP 403. This is exhaustive over the implemented operations on one file, not a user study, so the 98.6% of [4] (13 errors in 900 attempts by 50 users) is a different experiment. All 2,000 random single-bit changes to a 4 KiB AES-256-GCM ciphertext were rejected (tested with the Python cryptography library, the same algorithm as the browser), and a wrong key was refused.

TABLE III
PERMITTED OPERATIONS PER ROLE (OBSERVED LIVE)

Role	Read, list	Replace	Delete	Share, role, revoke, history
Owner	yes	yes	yes	yes
Editor	yes	yes	no	no
Viewer	yes	no	no	no
No role	no	no	no	no

We then edited an unmodified chain of 461 entries in nine ways (Fig. 6). Editing any field or the stored hash, deleting a middle entry, swapping neighbours and inserting a forged entry were detected in every trial (8 of 9 classes). Removing entries from the tail was not detected in any of 199 trials, and rewriting an edited entry together with every later hash passed verification, because the verifier checks only internal consistency and the chain has no signature or external anchor. A fully compromised database could therefore hide its tracks.

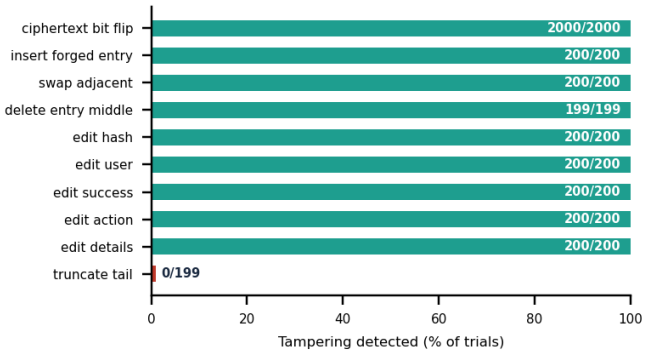


Fig. 6. Share of trials in which each audit or ciphertext tampering was detected (per-class counts shown).

Observed against the objectives, the encryption and hash checks cost little, the role check is exact over the operations tested, and the audit log needs work. Three remedies follow from the measurements. First, serialise the read-then-insert step of the audit writer, for example with a database transaction that locks the last row, so concurrent requests cannot fork the chain. Second, sign the chain head or write it to a store the application cannot modify, which would expose both tail truncation and a recomputed rewrite, as the designs of [13] and [14] do. Third, make verification incremental from a trusted checkpoint, because a full walk grows linearly and reached 404 ms at 5,000 entries.

E. Comparison with the Base Paper

The base paper's 8,506 µs per record is a calculation of attribute-based encryption from OpenSSL timings on a Raspberry Pi; CipherVault's 23.8 ms is a measured AES-GCM operation over a 10 MiB file in a browser. The workloads, hardware and metrics differ, so the figures cannot be ranked. Against the IJEDR paper [4], which reports +320 ms for a 10 MB upload, we measured about 14.4 ms of end-to-end overhead for 10 MiB, but its hardware, network and chunked, signed pipeline are undisclosed; likewise its 100% tamper detection covers 50 events and ours covers nine classes, one of which fails. Two design differences are certain: CipherVault uses 120,000 PBKDF2 iterations against 310,000, and has no multi-factor sign-in, chunking or ECDSA-signed log entries.

F. Failure Cases and Limits

Besides the unmasked tail truncation, 100 concurrent failed logins across four workers left the chain invalid at entry 5,047 of 5,145, because the writer reads the last hash and commits without a lock. Verification time grows linearly: 27.2 ms for 461 entries and 404 ms for 5,000. Revocation deletes the server's wrapped key but cannot recall a key or ciphertext already downloaded, and there is no re-keying. Replacing content reuses the same file key with a fresh random 96-bit IV, so the nonce-collision bound applies after very many updates [18]. The server delivers the JavaScript that performs the cryptography, file names, sizes and owners are plaintext metadata, a lost password means lost data, the token lives in sessionStorage, the audit list is visible to any signed-in user, and files above 25 MB are not supported because encryption is whole-file in memory. Firefox, Safari, PostgreSQL, multi-user load and assistive-technology audits were not tested.

V. CONCLUSION AND FUTURE SCOPE

CipherVault meets its six objectives: registration with bcrypt and JWT, AES-256-GCM encryption before upload, owner-and-role access control, SHA-256 verification on download, an audit log of uploads, downloads, deletions and denials, and a measured evaluation. Encryption cost is small next to the transfer itself (23.8 ms for 10 MiB), the role model matched in all 36 cells, and all 2,000 random bit flips in a test ciphertext were rejected. The main finding is about the audit trail: a plain hash chain detects edits, deletions and reordering but not truncation or a recomputed rewrite, and it breaks under concurrent writers. Limits are the shared measurement host, one browser, a 25 MB cap, no user study and the trust placed in the server's JavaScript. Future work in order of value: serialise audit writes and sign or externally anchor the chain head; chunked streaming for large files; key rotation on revoke; multi-factor sign-in; restricting the global audit list to the caller's own files; and testing on more browsers and under multi-user load.

REFERENCES

[1] A. Nekouie, M. Vafaei Jahan, M. H. Moattar and R. Sheibani, "Secure electronic health record access control via blockchain, dual-attribute encryption, and large language model-based attribute extraction," Scientific Reports, vol. 16, art. no. 8673, 2026, doi: 10.1038/s41598-026-39690-2.

[2] M. Backendal, M. Haller and K. G. Paterson, "MEGA: Malleable encryption goes awry," in Proc. IEEE Symp. Security and Privacy (SP), 2023, pp. 146-163.

[3] J. Bethencourt, A. Sahai and B. Waters, "Ciphertext-policy attribute-based encryption," in Proc. IEEE Symp. Security and Privacy (SP), 2007, pp. 321-334, doi: 10.1109/SP.2007.11.

[4] G. Sowmiya, R. Nivas and E. Vikram, "Secure cloud file storage architecture with client-side encryption and access control," Int. J. Eng. Develop. Res. (IJEDR), vol. 14, no. 2, pp. 576-579, Apr. 2026, paper IJEDR2602501.

[5] M. Watson, Ed., "Web Cryptography API," W3C Recommendation, Jan. 26, 2017. [Online]. Available: https://www.w3.org/TR/2017/REC-WebCryptoAPI-20170126/

[6] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in Proc. 13th EuroSys Conf., Porto, Portugal, Apr. 2018, doi: 10.1145/3190508.3190538.

[7] E.-J. Goh, H. Shacham, N. Modadugu and D. Boneh, "SiRiUS: Securing remote untrusted storage," in Proc. Network and Distributed System Security Symp. (NDSS), 2003.

[8] M. Kallahalla, E. Riedel, R. Swaminathan, Q. Wang and K. Fu, "Plutus: Scalable secure file sharing on untrusted storage," in Proc. 2nd USENIX Conf. File and Storage Technologies (FAST), San Francisco, CA, USA, 2003, pp. 29-42.

[9] Z. Wilcox-O'Hearn and B. Warner, "Tahoe: The least-authority filesystem," in Proc. 4th ACM Int. Workshop on Storage Security and Survivability (StorageSS), 2008, pp. 21-26, doi: 10.1145/1456469.1456474.

[10] E. Saginayev, A. Niyazov, A. Razaque, Zh. Kalpeyeva and A. Urazgaliyeva, "Secure file storage on cloud using hybrid cryptography," Computing & Engineering, vol. 2, no. 2, pp. 1-7, 2024, doi: 10.51301/ce.2024.i2.01.

[11] P. Thenmozhi, V. D. Saralesh and R. Sakthi, "Geocrypt: Location-based secure file encryption and access control," ITM Web of Conferences, vol. 82, art. no. 03019, 2026, doi: 10.1051/itmconf/20268203019.

[12] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn and R. Chandramouli, "Proposed NIST standard for role-based access control," ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224-274, Aug. 2001, doi: 10.1145/501978.501980.

[13] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," ACM Trans. Inf. Syst. Secur., vol. 2, no. 2, pp. 159-176, May 1999.

[14] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in Proc. 18th USENIX Security Symp., Montreal, QC, Canada, Aug. 2009, pp. 317-334.

[15] K. Moriarty, Ed., B. Kaliski and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," IETF RFC 8018, Jan. 2017.

[16] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[17] K. Moriarty, Ed., B. Kaliski, J. Jonsson and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," IETF RFC 8017, Nov. 2016.

[18] H. Bock, A. Zauner, S. Devlin, J. Somorovsky and P. Jovanovic, "Nonce-disrespecting adversaries: Practical forgery attacks on GCM in TLS," in Proc. 10th USENIX Workshop on Offensive Technologies (WOOT), Austin, TX, USA, Aug. 2016.

[19] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf., Monterey, CA, USA, Jun. 1999, pp. 81-92.

[20] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.